
WHITE PAPER · WORK, ADOPTION & JUDGMENT

The Exception Path Is the Real Automated Workflow

Design recovery before declaring a workflow automated

Marco Policani

Enterprise Portfolio · PMO · AI Operating Governance

policani.net · linkedin.com/in/marcpolicani

July 2026

EXECUTIVE SUMMARY

A workflow is automated only when its exception path is designed, staffed, and rehearsed, not merely when the routine cases run fast.

Automation leaves people the cases it could not handle, which by this paper's reasoning tend to be the hardest ones. The cases it hands back are where judgment, compliance exposure, and customer trust can concentrate, and where an automation plan can easily stop designing. This paper sets out what a workflow must do with those cases (sort them, contain them, route them to owners with context, recover fully, and learn) before "automated" is an honest description. The standard rests on reasoning and related research; no study tests it directly.

- 01** Replace the single manual-review queue with a list of the reasons processing stopped, such as missing data, low confidence, or a policy conflict. One undifferentiated queue hides which problems are frequent, urgent, or fixable.
- 02** Assign two owners to each kind of exception: one resolves the case, one fixes the cause that keeps producing cases like it.
- 03** Measure how many causes you have retired, not how much volume you have suppressed. A kind of exception that disappeared only because a threshold moved did not go away; its errors now land downstream.

The argument

When a team tells me their workflow is automated, I ask to see the exception queue, the list of cases the automation could not handle. The answer shows whether anyone owns them. What I have seen behind that request is a companion inbox, such as a shared mailbox, where everything the automation could not handle quietly piles up. No classification. No owners. No service expectation. No record of what was tried. The automation handles its routine volume well. The operation's actual risk, cost, and reputation live in the rest. That is an account of what I observed, not a measured rate. No source used here counts how common an unowned exception queue is, so treat

the request as a test to run on your own workflow. An owned, classified queue is a good answer to it.

The happy path, the cases that run straight through, proves throughput. The exception path, what happens to a case the automation cannot handle, proves whether the operation can detect what the automation cannot handle, route it to an owner, limit the damage, keep the evidence, recover, and learn. This paper's position is that a workflow should not be called automated until that second path exists, has owners, and has been rehearsed.

Why would the leftover cases be the hard ones? The reason comes from how automation is scoped, and it is this paper's reasoning, not a measured result. Automation is built for the cases that can be recognized and handled by rule or by model, and teams usually automate the easiest of those first. Whatever it takes out of human hands, the cases left tend to be the ones it could not handle or was never designed to handle: the ambiguous, the malformed, the conflicting, the novel, and the unsafe. A leftover case can be easy and rare, so this is a tendency, not a law. Where it holds, the exception stream (the flow of cases the automation hands back) is where the hardest judgment concentrates, where compliance exposure can hide, and where customer trust can be won or lost.

Lisanne Bainbridge made the matching point about industrial control rooms in 1983. A designer who automates what can be automated still leaves the operator the tasks the designer cannot think how to automate. When the operator must take over, something is probably wrong, so the job calls for more skill, and skills fade without practice.[1] Her examples were process plants and flight decks, so applying her point to business operations is an analogy. Human oversight can also be weaker than a design assumes: when people carry other work alongside, monitoring automation is error-prone, and their detection of its failures falls as the automation runs more reliably.[2] The exception path is where this problem is either designed for or discovered at the worst possible time.

This paper reverses the order that workflow design encourages. Workflow models map the expected paths first and cope less well with unexpected events.[3] Here the exception path is the real workflow, and the happy path is the optimization. It sets out a six-part path (classify, detect and contain, route with context, own the response, recover to a known state, learn and redesign) as the standard a workflow must meet before "automated" is an honest description. The six parts are the paper's own design. The sections below say what the related research supports, and where no study tests it. The paper is written for workflow owners, automation leaders, AI program owners, operations executives, and risk teams.

The economics of the leftover cases

Why does the exception path decide the economics? Because an exception tends to cost more and fail harder than a routine case. A routine case costs seconds of compute. An exception costs specialist minutes or hours, plus the routing tour to find the specialist, plus rebuilding the context once it arrives. A routine failure is a retry. A mishandled exception is a lapsed obligation, a wrong payment, an angry customer, or a regulator's letter. A small share of the volume can carry a large share of the consequence. This is reasoning from how the work divides, and no source used here prices the two kinds of case.

The gap widens as automation succeeds. Suppose automation raises the share of cases that run straight through from ninety to ninety-six percent, and that the easiest cases were automated first. The queue shrinks, but the cases still in it are now the hardest six percent, the ones the improved model still could not handle. Each gain in coverage leaves a harder set behind. That follows from the reasoning above and is not a measured trend. An operation that staffs its exception handling for last year's mix will find this year's mix harder per case. "The queue is smaller now" and "the queue is fine now" are different claims. Only the second one needs evidence.

Accounting hides the gap. Counts, speeds, and rates for the routine path can come straight from the pipeline. Exception metrics exist only when someone decides the queue is worth measuring. The dashboard then fills with the workflow's best news while its risk concentrates where the instruments are not. Consider a proposal that gives the exception path one sentence ("exceptions route to manual review"), staffs it as an afterthought, and leaves it unmeasured. Nothing on its dashboard would show the omission. Nobody decided to hide the queue. Nobody decided to look.

The six-part exception path

The six parts run in operating order, and each depends on the one before it. Detection needs the classes to know what it is looking for. Routing needs the class to pick the person. Ownership needs a route, recovery needs an owner who answers for the finish, and learning needs the recovery records. Reorder them and a part loses its input. Together they are the difference between an exception path and an exception pile.

EXHIBIT 1**The exception-recovery path****1. Classify**

Name why processing stopped and how urgent the case is.

2. Detect and contain

Know when the automation is out of its depth, and stop moving.

3. Route with context

A decision package, not an alarm. The reviewer acts without hunting.

4. Assign two owners

One resolves the case; another removes the cause that created it.

5. Recover

Every touched application corrected. Partial actions unwound.

6. Learn and redesign

Read the queue quarterly. Retire causes, not volume.

The research behind the parts is related, not direct. Russell, van der Aalst, and ter Hofstede sorted workflow exceptions into five event types, such as a missed deadline, an unavailable resource, or a violated rule. They described three choices for each: what happens to the stalled item, what happens to the rest of the case, and whether to do nothing, undo the effects, or make up for them.[3] They also note that a handler can be written only for an expected type of exception, which is why the first part below keeps a place for the novel case.[3] Their types describe events inside workflow software, and the first part's list describes why a business case stopped, so the two lists differ. NIST's AI risk-management framework treats monitoring, response, and recovery as ongoing work.[4] Microsoft's report on red-teaming 100 generative AI products says the work of securing AI systems will never be complete and that the human element is crucial.[5] Neither tests the six-part sequence.

Classify exceptions

The first part replaces the generic manual-review bucket with a taxonomy, a named list of the reasons ordinary processing was unsafe or impossible. Each reason is a class, a kind of exception. Missing or conflicting data. Low confidence. Policy conflict. A failed dependency. An unusual or unauthorized request. Suspected harm. The genuinely novel case. Each class implies a different urgency, a different skill, and a different fix at the source. That is exactly the information the single bucket destroys.

Classification converts the queue from a pile into a signal. Frequency by class shows where the automation's coverage actually ends. Severity by class shows where the risk lives. Recurrence by class shows which upstream fix would pay. The taxonomy also drives staffing, because a policy-conflict class needs someone with policy authority and a data-conflict class needs someone who can change the source applications. It also separates the routine cases from the contain-it-now cases, which no first-in-first-out mailbox can do. Read as one number, a hundred weekly exceptions hide this difference.

Build the classes from history, not imagination. Sample the last few hundred exceptions from the queue as it exists. Have the people who resolved them name the reasons in their own words, then group the names. The grouping should settle into a list short enough that each class can have an owner. If nearly every case is its own group, the groups are cases rather than causes. If two or three groups hold nearly everything, the sort is too coarse to tell anyone what to fix. Keep the leftover "novel" class. It is the list's most important member, not a gap in it, because novel cases are where tomorrow's classes announce themselves.

Detect and contain

The second part is the automation's obligation to know when it is out of its depth, and to stop moving when it is. Detection is knowing a case is outside what the automation can handle: input validation, confidence thresholds (a minimum score below which the automation must stop and hand the case over), policy checks, dependency health, anomaly signals. Containment is the safe state that follows: hold the case, release nothing partial, mark what was attempted, and report the event while response options are still open.

The failure this prevents is the plausible guess: the automation that, instead of declaring an exception, proceeds with its own best reading of a malformed input or a silently failed dependency. By this paper's reasoning, plausible guesses are among the costliest failures, because they turn detectable exceptions into undetected errors. Those errors show up later, downstream, stripped of the context that would have made them cheap to fix. A workflow that never raises exceptions has not shown it can handle them. More likely it is lucky, or quietly guessing, and the difference is discovered at audit. Generative AI adds a risk here. Research surveys describe fluent output that is unsupported or fabricated,[6] which is the plausible guess in its most convincing form.

Partial completion deserves its own design attention. An automation that runs three steps of five before failing has created a state no one designed. Containment means all-or-nothing behavior, or explicit checkpoints with known recovery steps. The proof that this part works is measurable. Detection rates on seeded bad inputs, meaning

bad test cases planted on purpose. Error rates at the edge of what the automation handles. Time from the first sign of trouble to a contained state. Also the count of partial actions found later, a number whose target is zero and whose actual value is the honesty test.

Route with context

The third part delivers the case to a human as a decision package, not an alarm. The reviewer needs what the automation knew: the input, what was attempted, why processing stopped, the affected records and parties, and the clock on the case. It is assembled at handoff, not reconstructed across four applications and two chat threads. Rebuilding context adds nothing to the decision, and the automation already had it. Dropping it is a design choice that can be reversed.

Routing follows the classes: each class maps to a role with the authority and skill to resolve it, with a fallback and an escalation clock. A tempting mistake is broadcast routing, the exception channel everyone can see and no one owns. It produces the bystander queue: visible to everyone, worked by no one, aging politely until someone important asks about a specific case.

The measures are queue mechanics. Could the reviewer act without hunting? How long until acceptance? How many clarification round-trips? How many misroutes? Also queue age by class, which is the executive view. A routine-data class aging a week is an efficiency problem. A suspected-harm class aging a day is an incident in progress.

Every routed case also carries two clocks: how long it may wait for acceptance, and how long resolution may take once accepted. Both are set by class consequence, both are visible, and both escalate automatically when breached. The clocks convert “someone will get to it” into a managed commitment.

Assign response ownership

The fourth part separates two jobs the mailbox blurs into one. Resolving this case. Fixing the cause that keeps producing cases like it. Case ownership belongs in the operation: someone answers for each item reaching resolution on time. Cause ownership belongs wherever the cause lives. The product team whose parser drops the field. The data owner whose feed conflicts. The policy owner whose rule contradicts practice.

Without the second owner, the operation clears cases indefinitely. The queue becomes a permanent institution with headcount, dashboards, and no mandate to shrink. The same class gets resolved heroically, week after week, while its cause sits untouched. The design is a standing loop. Recurrence reports by class flow to cause

owners. Fix commitments come back with dates. Classes that outlive their commitments also escalate as risks someone must sign for.

The signature matters. An exception class that recurs because fixing it is not worth the cost is a legitimate business decision: priced, owned, revisited. The same class recurring because no one with authority ever looked is an unpriced liability that happens to sit in an operations queue. The ownership design exists to make the two distinguishable.

Recover to a known state

The fifth part defines what “resolved” means, and it means more than “ticket closed.” Recovery restores the world. Data corrected in every application the case touched. Partial transactions unwound or completed. Downstream teams told where they used wrong outputs. Customer commitments repaired. The workflow returned to a state where ordinary processing can safely resume. A case closed by workaround, fixed in one application and left wrong in two others, is not resolved. It is deferred, with interest.

Recovery steps are per-class and rehearsed. Workflow research names the basic choices for work already done: do nothing, undo it (roll back), or make up for it (compensate).[3] The rollback for a partial payment run. The reconciliation for a duplicate-record class. The re-entry check before a contained case rejoins the flow. Rehearsal is what separates a procedure from a document. The first live run of an unrehearsed rollback is itself an incident, performed under pressure, on real data. The evidence: recovery time by class, reconciliation results, repeat-contact rates (a customer calling again is recovery failing in public), and leftover errors found by sampling across applications.

The re-entry check closes the loop on containment. Before a resolved case rejoins ordinary processing, something verifies that the cause that ejected it is actually fixed. Cases that bounce, from exception to resolution to re-entry and back to the same exception, are the signature of workaround-as-resolution. Their rate belongs on the same page as the recovery times.

Evidence preservation runs through recovery and beyond it: what the automation attempted, what the reviewer decided, and why, kept with the case. The record serves three masters at once. The audit that will eventually ask. The recurrence analysis that needs resolved cases as raw material. The next reviewer of a similar case, who inherits reasoning instead of reinventing it.

Learn and redesign

The sixth part turns the exception stream into the workflow's improvement budget. Every class is a bet about what to fix. Validation that would retire a bad-data class. A change at the source application that would end a conflict class. A policy clarification that would dissolve a policy-conflict class. A wider scope for the automation, if the evidence now supports it, or a narrower one, if the evidence now demands it. The queue, read quarterly, works as a product-management document for the automation.

This part's characteristic corruption is threshold gaming. Manual-review volume becomes a target, and the cheap way to hit it is raising confidence thresholds or reclassifying exceptions as acceptable outputs, declaring victory by redefining defeat. The defense is measuring causes retired rather than volume suppressed. A class that disappeared because validation now catches it upstream is a win. A class that disappeared because the threshold moved is a risk transfer to whoever meets the errors downstream.

Learning also feeds the test machinery. Resolved exceptions are pre-labeled hard cases, and the interesting ones belong in the workflow's standing test cases, so the next model or prompt change gets tested against the hard cases the operation has already paid for. An automation program that discards its exception history is throwing away the only test data that arrives pre-validated by consequence.

The exception stream is also a possible early signal of drift. Sculley and colleagues describe how machine-learning systems accrue hidden maintenance costs as data and the outside world shift.[7] They do not say that a rising exception stream marks that drift. That step is this paper's inference, and the quarterly review is where to test it against a workflow's own records.

AI-based automation likely widens the list of classes in one specific way. The low-confidence and suspected-harm classes grow, because a deterministic parser fails on malformed input, while a generative tool can also fail on well-formed input it merely handles badly, with output that looks complete. Confidence thresholds, sampling of accepted outputs, and downstream verification become detection instruments in their own right. The reviewer's job shifts toward evaluation. In specialized domains, Microsoft's red team says, that evaluation can be done accurately only by subject-matter experts.[5] The six parts hold unchanged.

The path as the readiness standard

Run the six parts together and the opening position becomes a test: a workflow is automated when its exception path is designed, staffed, measured, and rehearsed,

not when its happy path demonstrates throughput. The readiness review is a short written record kept by the workflow owner. It is produced before launch and again before any scale-up. It asks for six things. Show the taxonomy. Show containment behavior on seeded bad cases. Show a routed case's context package. Name the case and cause owners. Walk the rehearsed recovery for the worst class. Show last quarter's retired causes, where the workflow has run long enough to have a last quarter. A team that can do those six things has an automated workflow. A team that cannot has a demo with an inbox.

EXHIBIT 2

The readiness review, in six requests

Show the taxonomy

Frequency, severity, and recurrence by class, current, not archival.

Show containment

Behavior on seeded bad inputs, and partial actions trending to zero.

Show a handoff

One routed case's context package, complete at delivery.

Name the owners

The case owner and the cause owner for each class.

Walk the recovery

The rehearsed procedure for the worst class, not the document.

Show retired causes

Last quarter's retired causes, the number that shows the path shrinking.

A missing answer and a weak answer lead to different decisions. A missing answer holds the launch or the scale-up, and freezes expansion of a workflow that is already live, because nobody can say what happens to the cases the automation leaves. A weak answer, such as a taxonomy with one class or an owner without authority, lets the work continue under a volume cap until the gap closes.

The standard scales with consequence. A low-consequence workflow earns a light path: few classes, simple containment, one owner, modest measurement. A workflow touching money, customers, or regulated records earns the full design. What no workflow earns is the sentence "exceptions route to manual review" standing in for all six parts, because that sentence, unexamined, is where the shared mailbox comes from.

What the record should show

A working path leaves evidence anyone can inspect:

- A taxonomy with frequency, severity, and recurrence by class, current, not archival.
- Containment results on seeded inputs, and partial-action findings trending to zero.
- Context-complete handoffs and queue age by class, with escalation clocks honored.
- Case and cause owners named per class, with remediation commitments dated.
- Recovery rehearsals completed, and repeat-contact and re-entry-bounce rates.
- Causes retired per quarter, the number that shows the path shrinking itself.

Read the record quarterly with two questions. Where is the path absorbing consequence well? Where is it subsidizing a defect that should be fixed at the source? The first question defends the staffing. The second directs the roadmap. A review that asks only the first builds a better and better buffer around an unimproved machine: competent, expensive, and permanent.

The last number is the health of the whole design. A mature exception path should get quieter over time in the classes it has learned, and louder only at the frontier of new work: new case types, new volumes, new models. A path whose volume is flat across quarters with no retired causes is an operations team subsidizing an unimproved automation, however smoothly the subsidy runs.

Objections and limits

Not every rare event deserves bespoke automation. Some exceptions should stay with people, and some cases should be excluded from the workflow entirely. The aim is not zero exceptions but a deliberate path that limits consequence and makes learning possible.

The first objection is cost: a six-part design is gold-plating for a queue three people were handling fine. Test it against the queue itself. Classify one sample of the existing pile and price what turns up: the aged compliance items, the specialist hours lost rebuilding context, the workaround fixes quietly building reconciliation debt, the repeat contacts. If the sample finds little, the standard agrees with the objection. A low-consequence, low-volume workflow earns a light path, and the rigor goes where the queue is expensive. What the sample itself costs is set out under "What to do next."

The second objection is that better models will shrink the exception stream. They may, and the quarterly record will show it. The reasoning above says only that what the automation leaves is harder than what it takes, not that the stream stays large. The path is how a team would learn that the stream shrank, and what to do if it grew. A related claim, that the AI can triage its own exceptions, is partly right. It can sort cases and assemble the context package. It cannot be the owner, rehearse the recovery, or decide when to stop, and its sorting needs a sampled check by a named person.

The third objection is that this is ordinary exception handling and incident management under a new name. Much of it is. Workflow research was already cataloguing exception handling in 2006,[3] and IT service management separates restoring service from fixing the underlying problem. The paper's addition is narrower. It makes the exercised path a requirement for using the word "automated," and it separates the owner of a case from the owner of its cause.

The last is a limit on the evidence. No source used here measures how common unowned exception queues are, and none tests whether the six-part path reduces incidents, cost, or rework. Bainbridge studied industrial plants and flight decks. Parasuraman and Manzey's review draws mostly on laboratory studies. Russell and colleagues compared the workflow tools and languages of 2006. NIST and the Microsoft report give general direction. The standard is a design this paper argues for and a test any team can run on its own workflow. It is not a forecast of results.

What to do next

For a new workflow, the build order follows the evidence:

- Build the taxonomy from real historical cases and frontline interviews.
- Design detection, containment, routing, ownership, evidence, and recovery for each high-consequence class.
- Exercise the path with missing, ambiguous, conflicting, and unsafe inputs before launch.
- Review exception demand as a capacity, product, and operations signal.

The pre-launch exercise is where the path proves it exists. Feed the workflow its worst cases on purpose: missing fields, conflicting records, ambiguous categories, an unauthorized request, an unsafe output. Watch each one classify, contain, route, and recover from start to finish. The workflow owner runs it, with the people who would work the queue and one person for each cause it exposes. As a planning estimate, a small workflow takes days, not months. Expect a first exercise to find a

gap between the path as drawn and the path as built; a clean first run deserves a second look. A launch gate that requires the exercise report makes “exceptions route to manual review” an impossible sentence to ship.

For a workflow that is already live, start with the queue you already have. Open it, classify a sample, and triage by severity before designing anything. A first sort can turn up items that could not wait, such as an aged compliance notice. It may change how the program’s risk is priced. If it finds nothing urgent, that is also a result.

If exceptions do not sit in one queue, because people fix them in email, chat, or side spreadsheets, build the sample from the other end. Ask the people who do the fixing to list the cases they remember. Pull the workflow’s complaints, reversals, corrections, and repeat contacts. Add the seeded bad inputs from the exercise. If the records cannot tell the cases apart at all, that is the first finding: the workflow cannot yet show what it hands back, so it cannot yet be called automated.

The cost has three parts. As a planning estimate, a first sort of a few hundred cases is a matter of working days for the workflow owner and the people who work the queue, with a cause owner joining for each class that recurs. The standing cost is the quarterly review and the cause owners’ fix backlog. The larger cost is what the sort reveals: staffing and fixes the queue was never budgeted for. The companion paper “The Coordination Tax” sets the specialist hours each exception consumes against the hours the automation saves on routine cases, which is the comparison this cost belongs in.

Volume forecasting for the path belongs in every scale decision, and the arithmetic is unforgiving. Doubling happy-path volume roughly doubles exception volume at the current coverage, and the specialist pool does not double by wishing. A scale proposal should state expected exception load by class, the staffing that absorbs it, and the retirement work that will narrow it. If it does not, it is saying plainly that the plan is to let the queue age. Written down, that choice can be weighed against the savings.

Staff the path as a designed role, not a punishment posting. Exception handling done well is senior work: judgment on the hardest cases, spotting what they have in common, authority to change applications and policies. Treated as overflow labor, it can end up with the queue quality it was staffed for. The quarterly exception review, chaired by the workflow owner with cause owners present, is where the role’s findings become the program’s roadmap.

What “automated” has to mean

The redefinition is the point. Automated means the failure modes are governed, not merely that the success mode is fast. Design the path for the cases the automation cannot handle. Classify them by cause, contain them before consequence spreads, route them with context intact, give each class two owners, recover fully across every touched application, and mine the queue quarterly for the causes worth retiring. Then the happy path can be trusted with the routine cases, which is the work it was built for.

The readiness standard travels well, too. It works as a due-diligence question for vendor automations, as an audit lens for inherited workflows, and as a one-line policy for new builds: no launch without the exercised path.

Notes and sources

- [1] Lisanne Bainbridge, “Ironies of Automation,” *Automatica* 19(6), 1983, 775–779.
https://ckrybus.com/static/papers/Bainbridge_1983_Automatica.pdf
- [2] Raja Parasuraman and Dietrich H. Manzey, “Complacency and Bias in Human Use of Automation: An Attentional Integration,” *Human Factors* 52, no. 3 (2010): 381–410.
<https://depositonce.tu-berlin.de/bitstreams/cafd2873-814b-4c59-bab1-addd42e249d2/download>
- [3] Nick Russell, Wil M. P. van der Aalst, and Arthur H. M. ter Hofstede, “Workflow Exception Patterns,” in *Advanced Information Systems Engineering (CAiSE 2006)*, Lecture Notes in Computer Science 4001 (Springer, 2006), 288–302.
<https://www.vdaalst.com/publications/p325.pdf>
- [4] National Institute of Standards and Technology, “AI Risk Management Framework Core,” AI RMF 1.0, 2023. <https://airc.nist.gov/airmf-resources/airmf/5-sec-core/>
- [5] Blake Bullwinkel, Amanda Minnich, Shiven Chawla, Gary Lopez, Martin Pouliot, Whitney Maxwell, Joris de Gruyter, Katherine Pratt, Saphir Qi, Nina Chikanov, Roman Lutz, Raja Sekhar Rao Dheekonda, Bolor-Erdene Jagdagdorj, Eugenia Kim, Justin Song, Keegan Hines, Daniel Jones, Giorgio Severi, Richard Lundeen, Sam Vaughan, Victoria Westerhoff, Pete Bryan, Ram Shankar Siva Kumar, Yonatan Zunger, Chang Kawaguchi, and Mark Russinovich, “Lessons From Red Teaming 100 Generative AI Products,” Microsoft Research, January 2025. <https://www.microsoft.com/en-us/research/publication/lessons-from-red-teaming-100-generative-ai-products/>
- [6] Ziwei Ji, Nayeon Lee, Rita Frieske, et al., “Survey of Hallucination in Natural Language Generation,” *ACM Computing Surveys*, 2023.

<https://arxiv.org/abs/2202.03629>

[7] D. Sculley, Gary Holt, Daniel Golovin, et al., "Hidden Technical Debt in Machine Learning Systems," Advances in Neural Information Processing Systems 28 (NeurIPS 2015).

<https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf>

About the author

Marco Policani is an enterprise portfolio, PMO, and AI operating-governance leader. He builds portfolio governance systems where AI carries the analytical load and named humans own every decision — an approach documented across case studies, walkthroughs, and working governance guides on his portfolio site.

Portfolio & governance library: policani.net/governance · Full portfolio: policani.net · LinkedIn: linkedin.com/in/marcpolicani

This white paper may be shared freely with attribution. © 2026 Marco Policani.